Design and Simulation of a Real-Time Ticket Booking Management System Using a Linked-List-Based FIFO Queue

Title Page

Project Title	Design and Simulation of a Real-Time Ticket Booking Management System Using a Linked-List-Based FIFO Queue
Course Name	Data Structures Laboratory (DS LAB G1)
Submission Type	Project-Based Learning (PBL) Technical Report
Programme / Section	B.Tech. (AI & ML), `<Section / Semester>`
Student Name	`<Your Full Name>`
Student ID / Roll No.	`<Your ID>`
Course Instructor	Akash Bhardwaj
Institution	`<Your University / College Name>`
Date of Submission	`<DD Month YYYY>`
Implementation Language	C++ (ISO C++11, compiled with GCC / g++)
Repository	`<GitHub Repository URL>`

Table of Contents

1. Abstract
2. Introduction and Problem Statement
3. Data Structure Architecture and Design
4. Algorithmic Workflows (Textual Flowcharts)
5. Comparative Analysis: Memory and Performance
6. Conclusion and Future Scope
7. References

1. Abstract

Ticket reservation platforms must serve large numbers of concurrent requests while guaranteeing that customers are processed in the order of arrival. This project presents the design and simulation of a ticket booking management system built on a First-In, First-Out (FIFO) queue implemented from first principles as a singly linked list in C++. Standard library containers were deliberately excluded so that pointer mutation, dynamic allocation, and deallocation could be controlled explicitly. The queue maintains distinct `front` and `rear` pointers, which allows both enqueue and dequeue to execute in constant, O(1), time. A menu-driven console simulation supports ticket booking, processing, inspection, and display, with automatic ticket identifier generation and explicit handling of empty-queue conditions. The simulation demonstrates strict arrival-order service, absence of capacity overflow, and complete memory reclamation on termination. A comparative analysis shows that the linked representation trades a small per-node pointer overhead for elastic capacity and the elimination of resizing cost.

Keywords: Queue, FIFO, Singly Linked List, Dynamic Memory Allocation, Pointer Manipulation, Time Complexity, Ticket Booking Simulation.

---

2. Introduction and Problem Statement

2.1 Background

Online and counter-based ticket booking systems, whether for railways, airlines, cinemas, or live events, operate under conditions of highly variable demand. Request volumes may remain modest for extended periods and then surge by orders of magnitude when bookings open for a popular event. Under such conditions, the underlying software must satisfy three essential requirements: **fairness**, **scalability under load**, and **order preservation**.

2.2 Problem Statement

The core problem addressed in this project is stated as follows:

> *Design a booking-request management mechanism that accepts an unbounded and unpredictable stream of customer requests, preserves their exact order of arrival, serves them one at a time from the earliest, and performs each insertion and removal in constant time, without relying on pre-built container abstractions.*

The principal challenges are examined below.

(a) Fairness. Customers who submit requests earlier must be served earlier. Any mechanism that permits later requests to overtake earlier ones, whether through unordered storage or arbitrary selection, violates the fairness principle and erodes user trust.

(b) Handling High-Volume Traffic. Demand is bursty and cannot be predicted at design time. A system with a rigid capacity limit must either reject legitimate requests when saturated or reserve wasteful quantities of memory in anticipation of peak load. Neither outcome is acceptable in a well-engineered system.

(c) Order Preservation. The relative order of requests must remain invariant from the moment of arrival until the moment of service. This property must hold regardless of how many requests are inserted or removed in between.

(d) Operational Efficiency. Because every request incurs at least one insertion and one removal, the cost of these two operations dominates the system's throughput. Any linear-time (O(n)) overhead per operation would degrade performance catastrophically as the queue grows.

2.3 The FIFO Pipeline Requirement

The requirements above correspond precisely to the abstract behaviour of a **queue**, a linear Abstract Data Type (ADT) governed by the **First-In, First-Out (FIFO)** discipline. Formally, a queue exposes the following contract:

Operation	Semantics
`enqueue(x)`	Inserts element `x` at the **rear** (tail) of the sequence
`dequeue()`	Removes and returns the element at the **front** (head) of the sequence
`peek()`	Returns the front element without removing it
`isEmpty()`	Returns true if and only if the sequence contains no elements

The defining invariant is that *the element removed is always the element that has resided in the queue for the longest time.* Insertions occur exclusively at one end and removals exclusively at the other, which is exactly the temporal ordering required by a booking counter.

2.4 Objectives

1. To implement a FIFO queue from scratch using a singly linked list, without `std::queue`, `std::list`, or `std::vector`.
2. To demonstrate explicit pointer manipulation and manual dynamic memory management using `new` and `delete`.
3. To achieve O(1) time complexity for both `enqueue` and `dequeue`.
4. To simulate a ticket booking workflow through an interactive console interface with automatic ticket identifier generation.
5. To handle boundary conditions, namely empty-queue underflow and single-element removal, robustly and without memory faults.
6. To analyse the memory and performance trade-offs of the linked representation relative to an array-based alternative.

---

3. Data Structure Architecture and Design

3.1 Rationale for a Custom Singly Linked List

3.1.1 Why Not a Static Array?

A statically allocated array-based queue has a capacity fixed at compile time or at construction. This design suffers from several structural limitations:

- **Overflow risk.** Once the array is full, further insertions must be refused, which is unacceptable when the arrival rate is unpredictable.
- **Wasted memory.** To reduce overflow risk, the array must be over-provisioned, leaving most of the allocated storage unused during periods of low demand.
- **Shifting cost.** A naive array queue that keeps its front at index 0 must shift all remaining elements after each dequeue, an O(n) operation. This can be avoided only by adopting a circular buffer, which adds index-arithmetic complexity (`(rear + 1) % capacity`) and the problem of distinguishing a full queue from an empty one.
- **Resizing overhead.** Even a dynamically resizing array incurs an O(n) copy whenever its capacity is exceeded, producing latency spikes at the worst possible moment, namely during peak load.

3.1.2 Why Not a Standard Library Wrapper?

Containers such as `std::queue` provide a convenient interface but conceal their implementation. For an academic data structures project, the purpose is not merely to *use* a queue but to *understand and construct* one. Implementing the structure manually makes the following concepts explicit and verifiable:

- how nodes are allocated on the heap and how their lifetime is controlled,
- how a chain of nodes is formed and traversed through pointers,
- how the head and tail references must be kept consistent after every mutation,
- how memory is returned to the system to prevent leaks and dangling references.

3.1.3 Why a Singly Linked List Specifically?

A queue requires traversal in one direction only, from the front towards the rear. A **singly** linked list therefore provides all necessary connectivity with the minimum storage cost, a single `next` pointer per node. A doubly linked list would add a second pointer per node without contributing any capability that the FIFO discipline requires.

3.2 Anatomy of a Node

Each waiting customer is represented by one heap-allocated **node**. A node couples the *payload* (the booking data) with the *link* (the pointer to its successor).

```
struct Node {
    int         ticketId;       // Auto-generated unique ticket identifier
    std::string passengerName;  // Name of the passenger / customer
    std::string destination;    // Requested destination (additional payload)
    Node*       next;           // Link to the successor node (towards the rear)
};
```

Field	Type	Role

`ticketId`	`int`	Unique identifier, generated sequentially (starting at 1001) so that the user never enters it manually
`passengerName`	`std::string`	Identifies the customer to whom the ticket belongs
`destination`	`std::string`	Additional payload describing the requested journey
`next`	`Node*`	Pointer to the next node in the chain; `nullptr` for the last node

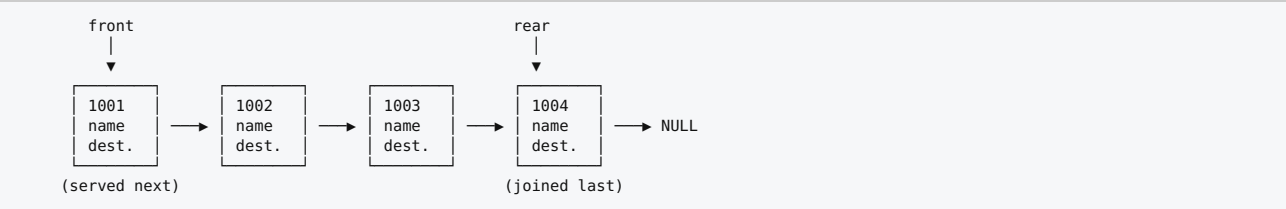
The constructor initialises `next` to `nullptr`. Consequently, every newly created node is a well-formed tail node: it terminates the list until some other operation links a successor to it.

3.3 The `front` and `rear` Pointers

The `TicketQueue` class manages the list through two private pointers and an element counter.

Member	Type	Role
`front`	`Node*`	Points to the **oldest** ticket, the next to be served. Governs deletion.
`rear`	`Node*`	Points to the **newest** ticket, the most recently booked. Governs insertion.
`count`	`int`	Number of pending tickets, maintained for O(1) size queries

The structure may be visualised as follows:



Class invariants. The following conditions hold before and after every public operation:

1. `front == nullptr` **if and only if** `rear == nullptr` **if and only if** `count == 0`.
2. If the queue is non-empty, then `rear->next == nullptr`.
3. Following `next` links from `front` visits exactly `count` nodes and terminates at `rear`.

Invariant 1 is the source of the most common implementation defect in linked queues: when the final node is dequeued, `front` becomes `nullptr`, and `rear` **must** be reset as well. Otherwise `rear` is left as a dangling pointer to freed memory, and the next enqueue would write through it.

3.4 Achieving O(1) Time Complexity

A singly linked list that retains only a `head` pointer can insert at the head in constant time but must traverse the entire list to append at the tail, costing O(n). The use of a dedicated **`rear` pointer** removes this limitation.

Enqueue in O(1). Insertion involves a fixed number of steps independent of the queue length n: one heap allocation, one pointer assignment (`rear->next = newNode`), and one pointer update (`rear = newNode`). No traversal occurs, because `rear` already references the tail.

Dequeue in O(1). Removal involves one temporary pointer assignment, one advance of `front`, an optional reset of `rear`, and one `delete`. No traversal and no element shifting occur, because `front` already references the head.

Because each operation executes a bounded number of primitive steps regardless of n, both are **O(1)**. The complete complexity profile is summarised below.

Operation	Time Complexity	Auxiliary Space	Justification
`enqueue`	**O(1)**	O(1)	Direct tail access through `rear`
`dequeue`	**O(1)**	O(1)	Direct head access through `front`
`peek`	**O(1)**	O(1)	Reads `front` only
`isEmpty`	**O(1)**	O(1)	Compares `front` with `nullptr`
`displayQueue`	O(n)	O(1)	Must visit each of the n nodes
Destructor	O(n)	O(1)	Must release each of the n nodes
Total storage		**O(n)**	One node per pending ticket

Total storage	—	**O(n)**	One node per pending ticket

(Strictly, O(1) refers to the cost of the queue logic itself; the cost of the heap allocator behind `new` and `delete` is implementation-defined and is treated as constant in the standard analysis.)

3.5 Memory Management Strategy

The implementation follows a strict ownership discipline:

- **Allocation:** each node is created with `new` at enqueue time, so memory consumption is proportional to the number of pending tickets.
- **Deallocation:** each node is destroyed with `delete` immediately after it is dequeued, so served tickets consume no memory.
- **Destructor:** on program termination, the `TicketQueue` destructor iterates through any remaining nodes and deletes them, guaranteeing leak-free shutdown.
- **Copy suppression:** the copy constructor and copy-assignment operator are explicitly deleted. Without this, a shallow copy would cause two queue objects to own the same nodes, resulting in a double `delete`.

---

4. Algorithmic Workflows (Textual Flowcharts)

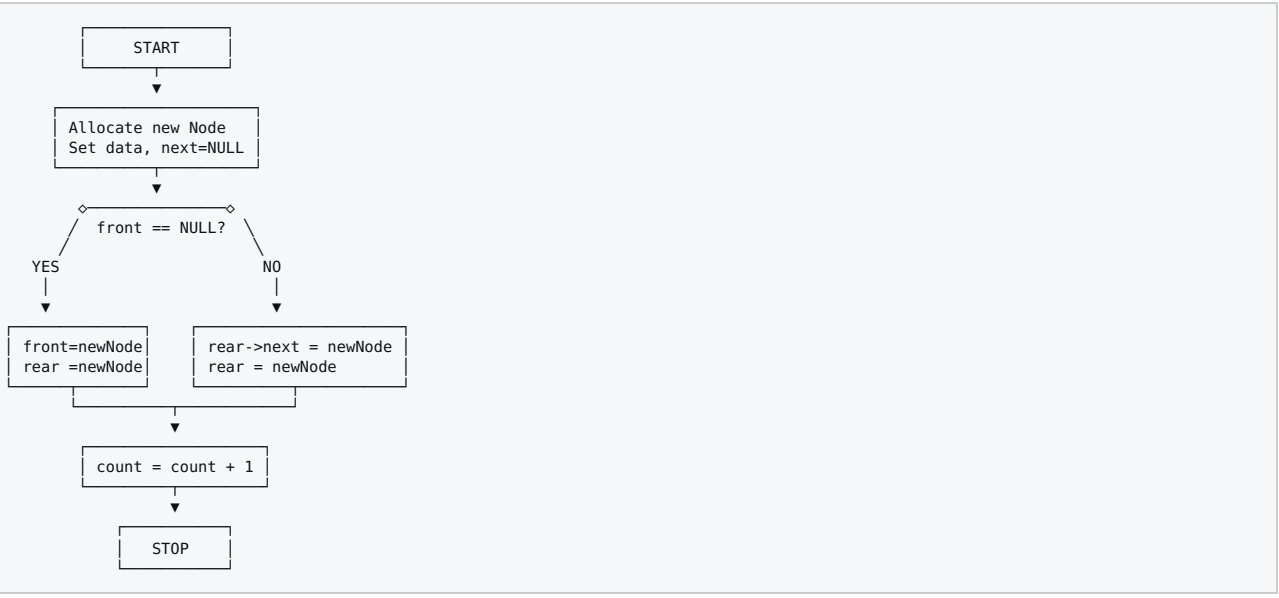
4.1 Enqueue Operation

Purpose: insert a new ticket at the rear of the queue. **Pre-condition:** none (the queue may be empty or non-empty). **Post-condition:** the new node is the last node; `rear` references it; `count` is increased by one.

```
ALGORITHM Enqueue(queue, id, name, destination)
──────────────────────────────────────────────────────────────
INPUT : queue       - the TicketQueue with pointers front and rear
        id          - auto-generated ticket identifier
        name        - passenger name
        destination - requested destination
OUTPUT: queue with a new node appended at the rear

 1.  START
 2.  newNode ← ALLOCATE a new Node on the heap          // new Node(...)
 3.  newNode.ticketId      ← id
 4.  newNode.passengerName ← name
 5.  newNode.destination   ← destination
 6.  newNode.next          ← NULL                       // tail node terminates the list
 7.  IF queue.front = NULL THEN                         // Case A: queue is empty
 8.        queue.front ← newNode                        //   new node becomes the head
 9.        queue.rear  ← newNode                        //   ...and also the tail
10.  ELSE                                               // Case B: queue is non-empty
11.        queue.rear.next ← newNode                    //   link old tail to new node
12.        queue.rear      ← newNode                    //   advance tail pointer
13.  END IF
14.  queue.count ← queue.count + 1
15.  DISPLAY confirmation (ticket ID and queue position)
16.  STOP
──────────────────────────────────────────────────────────────
TIME COMPLEXITY  : O(1)   (no loops, no traversal)
MEMORY EFFECT    : one new heap block allocated
```

Textual flowchart:



Pointer trace (non-empty queue, enqueue ticket 1003):

Stage	State
Before	`front → [1001] → [1002] → NULL`, `rear → [1002]`
After allocation	A detached node `[1003]` exists with `next = NULL`
After `rear->next = newNode`	`front → [1001] → [1002] → [1003] → NULL`, `rear` still at `[1002]`
After `rear = newNode`	`rear → [1003]`; the invariant `rear->next == NULL` is restored

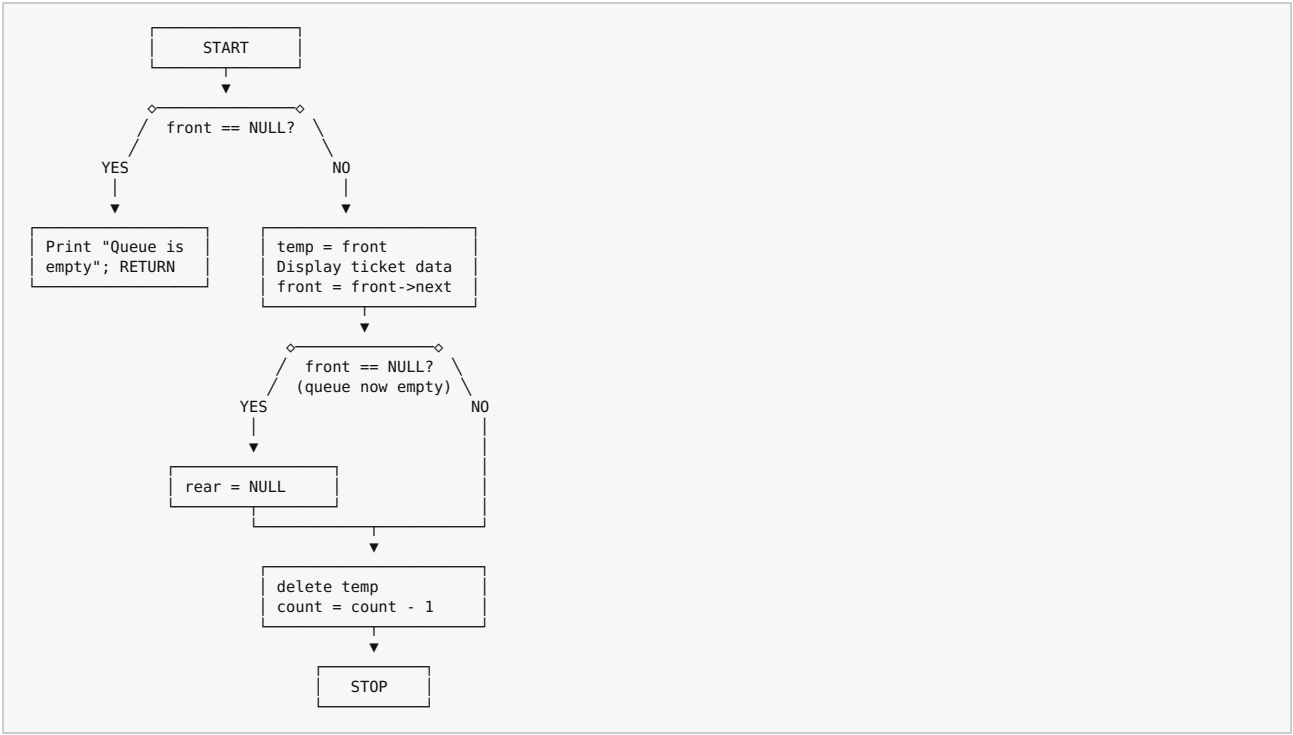
4.2 Dequeue Operation

Purpose: remove and process the ticket at the front of the queue. **Pre-condition:** the queue should be non-empty (the algorithm handles underflow explicitly). **Post-condition:** the former head node is released; `front` references its successor; `count` is decreased by one.

```
ALGORITHM Dequeue(queue)
──────────────────────────────────────────────────────────────
INPUT : queue - the TicketQueue with pointers front and rear
OUTPUT: queue with the front node removed and its memory freed

 1.  START
 2.  IF queue.front = NULL THEN                        // Underflow check
 3.        DISPLAY "Queue is empty: no tickets to process"
 4.        RETURN                                      // nothing is modified
 5.  END IF
 6.  temp ← queue.front                                // save reference to old head
 7.  DISPLAY ticket details held in temp               // confirm the booking
 8.  queue.front ← queue.front.next                    // advance head to successor
 9.  IF queue.front = NULL THEN                        // list has just become empty
10.        queue.rear ← NULL                          // prevent dangling rear pointer
11.  END IF
12.  FREE temp                                        // delete temp; return memory
13.  temp ← NULL                                      // avoid stale local pointer
14.  queue.count ← queue.count - 1
15.  STOP
──────────────────────────────────────────────────────────────
TIME COMPLEXITY  : O(1)   (no loops, no traversal)
MEMORY EFFECT    : one heap block released
```

Textual flowchart:



Pointer trace (dequeue from `[1001] → [1002] → NULL`):

Stage	State
Before	`front → [1001] → [1002] → NULL`, `rear → [1002]`
After `temp = front`	`temp → [1001]`, `front → [1001]`
After `front = front->next`	`front → [1002]`; `[1001]` is reachable only through `temp`
After `delete temp`	`[1001]` is returned to the heap; `front → [1002] → NULL`, `rear → [1002]`

Critical edge case (single-element queue): if only `[1002]` remained, step 8 would set `front = NULL`. Without step 10, `rear` would continue to point to `[1002]` after step 12 freed it. A subsequent `enqueue` would then execute `rear->next = newNode` on freed memory, producing undefined behaviour. Step 10 eliminates this defect.

4.3 Supporting Operations (Summary)

Peek: if `front = NULL`, report an empty queue; otherwise read and display the fields of `front` without modifying any pointer. *O(1).*

Display: if `front = NULL`, report an empty queue; otherwise set `current ← front` and, while `current ≠ NULL`, print its fields and set `current ← current.next`. A separate traversal pointer is used so that `front` itself is never altered. *O(n).*

---

5. Comparative Analysis: Memory and Performance

5.1 Array-Based Queue versus Linked-List-Based Queue

Parameter	Array-Based Queue	Linked-List-Based Queue
Memory Allocation	Contiguous block allocated up front (static) or reallocated on growth (dynamic).	Non-contiguous; one node allocated per element at the moment of insertion.
Memory Utilisation	Unused slots remain reserved while idle, causing internal fragmentation.	Exactly *n* nodes exist for *n* elements; no unused reserved capacity.
Overflow Risk	Overflow occurs when the array is full (fixed capacity).	No logical overflow; bounded only by available heap memory.
Underflow Handling	Requires a check on indices or a size counter.	Requires a check that `front == nullptr`.
Resizing Overhead	Dynamic arrays incur an O(n) copy whenever capacity is exceeded; amortised O(1) but with latency spikes.	None; the structure grows and shrinks node by node.

Pointer Tracking Overhead	None per element; only index variables (`front`, `rear`) are needed.	One `next` pointer per node (typically 8 bytes on a 64-bit system), plus the discipline of keeping `front` and `rear` consistent.
Enqueue Time	O(1) with a circular buffer; amortised O(1) with resizing.	O(1) worst case.
Dequeue Time	O(1) with a circular buffer; O(n) if elements are shifted.	O(1) worst case.
Memory Locality / Cache Behaviour	Excellent, because contiguous storage favours spatial locality and hardware prefetching.	Poorer, because nodes may be scattered across the heap.
Allocator Cost	One allocation (or occasional reallocation) overall.	One `new` and one `delete` per element, each carrying allocator overhead.
Memory Leak Risk	Low; a single block is released.	Higher; every node must be released, so ownership must be disciplined.
Dangling-Pointer Risk	Not applicable.	Present (for example, a stale `rear` after the last dequeue) and must be guarded against.
Implementation Complexity	Lower for the basic version; moderate for a circular version.	Moderate; requires careful pointer handling.
Suitability for Unpredictable Load	Poor to moderate.	**High.**

5.2 Quantitative Memory Model

Let S denote the size of the booking payload in bytes (`ticketId`, `passengerName`, `destination`) and P the size of a pointer.

- **Linked-list queue:** memory ≈ $n \times (S + P) + (2P + \text{sizeof}(\texttt{count}))$ for the control block.
- **Fixed array queue of capacity C:** memory ≈ $C \times S$, irrespective of how many of the C slots are occupied.

The linked representation consumes less memory than the fixed array whenever

$n \times (S + P) < C \times S$, that is, whenever $n < C \times S / (S + P)$.

Hence, at low or moderate occupancy the linked queue is the more economical structure, while the per-node pointer overhead of P bytes becomes relatively more significant only when the payload S is small. In a ticketing system where the payload includes at least two strings, S is substantially larger than P, so the proportional overhead of the pointer is modest.

5.3 Discussion

The analysis indicates a clear trade-off. The array-based queue offers superior cache locality and zero per-element pointer cost, advantages that matter when the maximum load is strictly bounded and known in advance. A ticketing system, however, is characterised by **bursty, unbounded, and unpredictable demand**. In that setting:

1. The overflow risk of a fixed array is a functional failure (a legitimate customer is turned away), whereas the pointer overhead of a linked list is merely a quantitative cost.
2. Over-provisioning an array to avoid overflow wastes memory during the long periods of low demand.
3. The resizing latency of a dynamic array occurs precisely at peak load, when responsiveness matters most.

The linked-list queue therefore represents the more appropriate design for this problem domain. The additional pointer overhead and the demand for rigorous memory discipline are accepted as the price of elastic capacity and predictable worst-case O(1) operations.

---

6. Conclusion and Future Scope

6.1 Conclusion

This project designed and implemented a ticket booking management simulation around a custom singly linked-list FIFO queue. The following conclusions are drawn.

1. **Correct FIFO semantics.** Maintaining a `front` pointer for removal and a `rear` pointer for insertion guarantees that tickets are served strictly in arrival order, satisfying the fairness and order-preservation requirements.
2. **Constant-time operations.** Both `enqueue` and `dequeue` execute in O(1) worst-case time, since neither requires traversal or element shifting.

3. **Elastic capacity.** Dynamic node allocation removes any fixed capacity limit, so the system cannot suffer logical overflow and does not reserve unused memory.
4. **Memory safety.** Explicit `delete` on every dequeue, the destructor sweep at shutdown, the reset of `rear` when the queue empties, and the deletion of copy operations together prevent memory leaks, dangling pointers, and double frees.
5. **Robust boundary handling.** Empty-queue underflow and single-element removal are handled explicitly, and the interactive interface validates user input.
6. **Justified trade-offs.** The comparative analysis shows that the linked representation accepts a per-node pointer overhead and weaker cache locality in return for flexibility and the absence of resizing cost, a favourable exchange for a bursty booking workload.

The project consolidates core data structures competencies: abstract data type design, pointer manipulation, dynamic memory management, and asymptotic analysis.

6.2 Future Scope

(1) Priority Queue for VIP and Emergency Tickets. The current design is strictly chronological. A priority-aware variant, implemented either as a sorted linked list (O(n) insertion, O(1) removal) or as a binary heap (O(log n) for both operations), would let premium, senior-citizen, or emergency bookings be served ahead of standard requests while preserving FIFO order within each priority level.

(2) Doubly Linked List for Bidirectional Navigation and Cancellation. Adding a `prev` pointer to each node would permit O(1) removal of an arbitrary node given a reference to it, enabling **ticket cancellation** from the middle of the queue, as well as backward traversal for auditing. The cost is an additional pointer per node and more intricate pointer updates.

(3) Persistence and Multi-Counter Simulation. Serialising the queue to a file would let pending bookings survive program restarts. A multi-counter extension, with several queues served in parallel and a load-balancing policy (for example, join the shortest queue), would model real booking halls more faithfully. Timestamps on nodes would additionally allow average waiting time and throughput analysis.

(4) Generic Programming and Concurrency (Optional). Converting the queue into a class template (`template <typename T>`) would allow reuse for related problems such as print-job spooling. Introducing mutex-guarded or lock-free operations would extend the design towards genuinely concurrent, real-time request handling.

---

7. References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
2. M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Pearson, 2013.
3. E. Horowitz, S. Sahni, and S. Anderson-Freed, *Fundamentals of Data Structures in C*, 2nd ed. Universities Press, 2008.
4. B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
5. ISO/IEC 14882:2011, *Programming Language C++*.
6. `<Add your course lecture notes / lab manual reference here>`

End of Report